

## 实验3 TINY扩充语言的语法树生成

## 一、实验内容：

## (一) 为Tiny语言扩充的语法有

1. 实现改写书写格式的新if语句；
2. 扩充算术表达式的运算符：++（前置自增1）、--（前置自减1）运算符（类似于C语言的++和--运算符，但不需要完成后置的自增1和自减1）、求余%、乘方^；
3. 扩充扩充比较运算符：<（小于）、>（大于）、<=（小于等于）、>=（大于等于）、<>（不等于）等运算符；
4. 增加正则表达式，其支持的运算符有：或(|)、连接(&)、闭包(#)、括号()、可选运算符(?)和基本正则表达式。
5. for语句的语法规则（类似于C语言的for语言格式）：  
书写格式：for(循环变量赋初值;条件;循环变量自增或自减1) 语句序列
6. while语句的语法规则（类似于C语言的while语言格式）：  
书写格式：while(条件) 语句序列 endwhile

## (二) 对应的语法规则分别为：

1. 把TINY语言原有的if语句书写格式  
if\_stmt--> if exp then stmt-sequence end | if exp then stmt-sequence else stmt-sequence end  
改写为：  
if\_stmt--> if(exp) stmt-sequence else stmt-sequence | if(exp) stmt-sequence
2. ++（前置自增1）、--（前置自减1）运算符、求余%、乘方^等运算符的文法规则请自行组织。
3. <（小于）、>（大于）、<=（小于等于）、>=（大于等于）、<>（不等于）等运算符的文法规则请自行组织。
4. 为tiny语言增加一种新的表达式——正则表达式，其支持的运算符有：或(|)、连接(&)、闭包(#)、括号()、可选运算符(?)和基本正则表达式，对应的文法规则请自行组织。
5. 为tiny语言增加一种新的语句，ID==正则表达式（同时增加正则表达式的赋值运算符==）
6. 为tiny语言增加一个符合上述for循环的书写格式的文法规则，
7. 为tiny语言增加一个符合上述while循环的书写格式的文法规则，
8. 为了实现以上的扩充或改写功能，还需要注意对原tiny语言的文法规则做一些相应的改造处理。

Tiny语言原来的文法规则，可参考：云盘中参考书P97及P136的文法规则。

## 二、实验要求：

- (1) 要提供一个源程序编辑的界面，以让用户输入源程序（可输入、可保存、可打开源程序）
- (2) 可由用户选择是否生成语法树，并可查看所生成的语法树。
- (3) 实验3的实现只能选用的程序设计语言为：C++
- (4) 要求应用程序的操作界面应为Windows界面。
- (5) 应该书写完善的软件文档

## 三、完成时间：四周（第9周-第13周）

## 四、上交方法：

通过砺儒云平台进行实验提交

## 五、完成方式：每个学生自行独立完成。

## 六、测试数据（语法正确和错误的源程序均要有测试文件）

测试文件1：（该程序中的if语句还是采用原Tiny语言中if语句的书写格式进行书写，作为实验3的测试源程序，你需要根据实验3实际要求的改写方式进行改写）

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }
if 0<x then { don't compute if x <= 0 }
  for( fact := 1; x>0;--x)
    fact := fact * x
  write fact { output factorial of x }
end
```

测试文件2：（该程序中的if语句还是采用原Tiny语言中if语句的书写格式进行书写，作为实验3的测试源程序，你需要根据实验3实际要求的改写方式进行改写）

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }
if 0<x then { don't compute if x <= 0 }
fact := 1;
while( x>0 )
  fact := fact * x;
  --x
endwhile
write fact { output factorial of x }
end
```

测试文件3：(该程序中的if语句还是采用原Tiny语言中if语句的书写格式进行书写，作为实验3的测试源程序，你需要根据实验3实际要求的改写方式进行改写)

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }

if x>0 then { don't compute if x <= 0 }
  fact := 1;
  repeat
    fact := fact * x;
    --x
  until x = 0;
  write fact { output factorial of x }
end
```

为了做好全面的测试，你还需要增加更多的其他相应的测试文件。